



Eksempel: Vandstandsstyring

*PLC + **snap7** + CSV-logging — et fuldt gennemganger eksempel*

Kravspeg. → Design → Kodning → Test

Formål med dette dokument

Dette dokument viser, hvordan V-modellens processer bruges i praksis. Vi følger et fuldt eksempel fra kravspecifikation til accepttest for et **pumpeanlæg med vandstandsmåling**.

- ✓ Skriv en kravspecifikation med K0x-krav og acceptkriterier
- ✓ Tegn blokdiagram og state machine
- ✓ Implementer Python-kode med **snap7**, docstrings og CSV-logging
- ✓ Gennemfør og dokumentér FAT og accepttest

Indhold

1	Systembeskrivelse	2
1.1	Formål	2
1.2	Projektomfang	2
1.3	Komponenter	2
2	Kravspekifikation	2
2.1	Funktionelle krav	2
2.2	Ikke-funktionelle krav	3
3	Systemdesign	3
3.1	Blokdiagram	3
3.2	DB1 — datablok-struktur	3
4	Moduldesign	4
4.1	State machine — Python-script	4
4.2	Flowchart — Python-hovedløkke	5
5	Kodning	6
6	Testdokumentation	10
6.1	Factory Acceptance Test (FAT)	10
6.1.1	Performance-kontrol (NK01)	10
6.2	Accepttest (UAT)	11
7	Eksempel på CSV-output	11
8	Opsummering — processen kort	13

1 | Systembeskrivelse

1.1 Formål

Systemet skal holde vandstanden i en tank inden for et ønsket interval ved at styre en pumpe via en Siemens S7-1200 PLC. En Python-applikation på en PC forbinder til PLC'en via Snap7 over Ethernet, læser vandstand og pumpestatus hvert sekund og logger data til en CSV-fil.

1.2 Projektomfang

Hvad er med og hvad er ikke med

Er med:

- Automatisk start/stop af pumpe baseret på vandstand
- Alarmhåndtering ved for højt niveau eller kommunikationsfejl
- Kontinuerlig datalogning til CSV med tidsstempel

Er ikke med:

- HMI-visualisering (kan tilføjes i et efterfølgende trin)
- Fjernalarmering (SMS, e-mail)

1.3 Komponenter

Komponent	Type	Tilslutning	Bemærkning
PLC	Siemens S7-1200	—	Styreenhed
Niveau-sensor	Digital, <i>NC</i>	I0.0	Aktivt når niveau < 20 %
Overflow-switch	Digital, <i>NC</i>	I0.1	Aktivt ved overløb
Pumpe	Digital output	Q0.0	24 V DC relæ
PC (Python)	Ethernet	DB1	Snap7-klient

2 | Kravspecifikation

2.1 Funktionelle krav

Læs inden du bruger kravene

Hvert krav har et unikt ID (K0x), en beskrivelse og et **acceptkriterie**. Her betyder acceptkriteriet den målbare observation, du bruger til at **verificere** kravet i FAT eller systemtest.

Det er **ikke** det samme som den afsluttende accepttest i V-modellen. Accepttesten (UAT) ligger til sidst og bruges til at **validere**, at den samlede løsning giver mening for brugeren i praksis.

ID	Krav	Acceptkriterie
K01	Pumpen skal starte automatisk, når vandstanden falder til under 20 %.	Python setter DB1.DBX0 bit 0 = 1, og Q0.0 = TRUE bekræftes i PLC.
K02	Pumpen skal stoppe automatisk, når vandstanden når 80 %.	Python setter DB1.DBX0 bit 1 = 1, og Q0.0 = FALSE bekræftes.
K03	Systemet skal gå i ALARM-tilstand ved overflow (I0.1 = 1).	DB1.DBX6 sættes til 1 af PLC'en, Python skifter til ALARM-state og stopper pumpem.
K04	Systemet skal logge vandstand, pumpestatus og alarmkode til CSV hvert sekund.	CSV-fil indeholder rækker med maks. 1 s mellemrum og korrekte kolonner.
K05	Systemet skal håndtere Snap7-fejl uden at gå ned — fejl skal logges og forbindelsen genoprettes.	Ved netværksafbrud prøver scriptet igen hvert 5 s og logger fejlen til CSV.

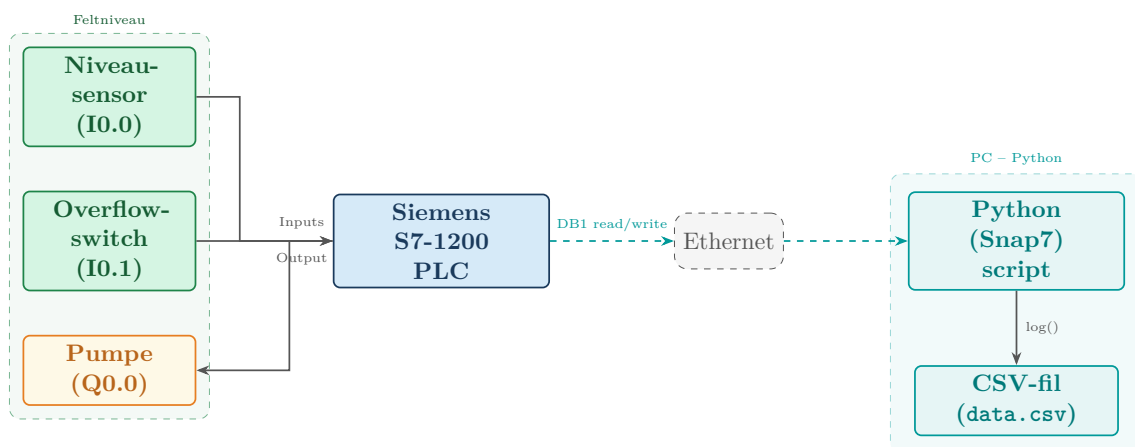
2.2 Ikke-funktionelle krav

ID	Krav
NK01	Opdateringsinterval må ikke overstige 1,5 s under normal drift.
NK02	CSV-filnavn dannes automatisk ud fra opstartstidspunkt (format: YYYY-MM-DD_HHMMSS.csv).
NK03	Python-scriptet skal have kommentarer der forklarer hver logisk del.

3 | Systemdesign

3.1 Blokdiagram

Blokdiagrammet viser systemets hoveddele og dataflow. Sensorer og aktuatorer er tilkoblet PLC'en. PC'en kommunikerer med PLC'en via Snap7 over Ethernet og læser/skriver data i DB1.



3.2 DB1 — datablok-struktur

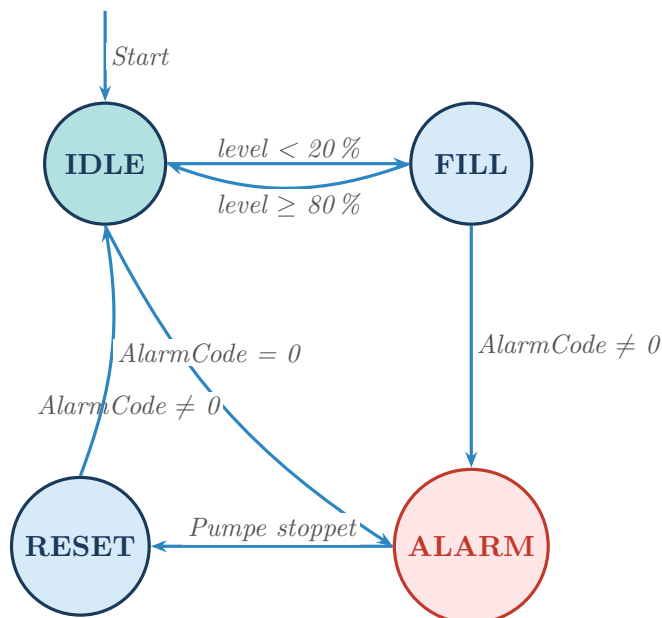
PLC'en eksponerer en datablok (DB1) som Python-scriptet læser og skriver til.

Offset	Snap7 addr.	Type	Navn	Beskrivelse
DBX0	byte 0, bit 0	BOOL	PumpStart	Python skriver 1 for at starte pumpe
DBX0	byte 0, bit 1	BOOL	PumpStop	Python skriver 1 for at stoppe pumpe
DBX1	byte 1, bit 0	BOOL	PumpRunning	PLC skriver 1 når pumpe kører
DBX1	byte 1, bit 1	BOOL	PumpFault	PLC skriver 1 ved motorværnsfejl
DBX2	byte 2–5	REAL	WaterLevel	Vandstand i % (0.0–100.0)
DBX6	byte 6–7	INT	AlarmCode	0=OK, 1=Overflow, 2=CommFault

4 | Moduldesign

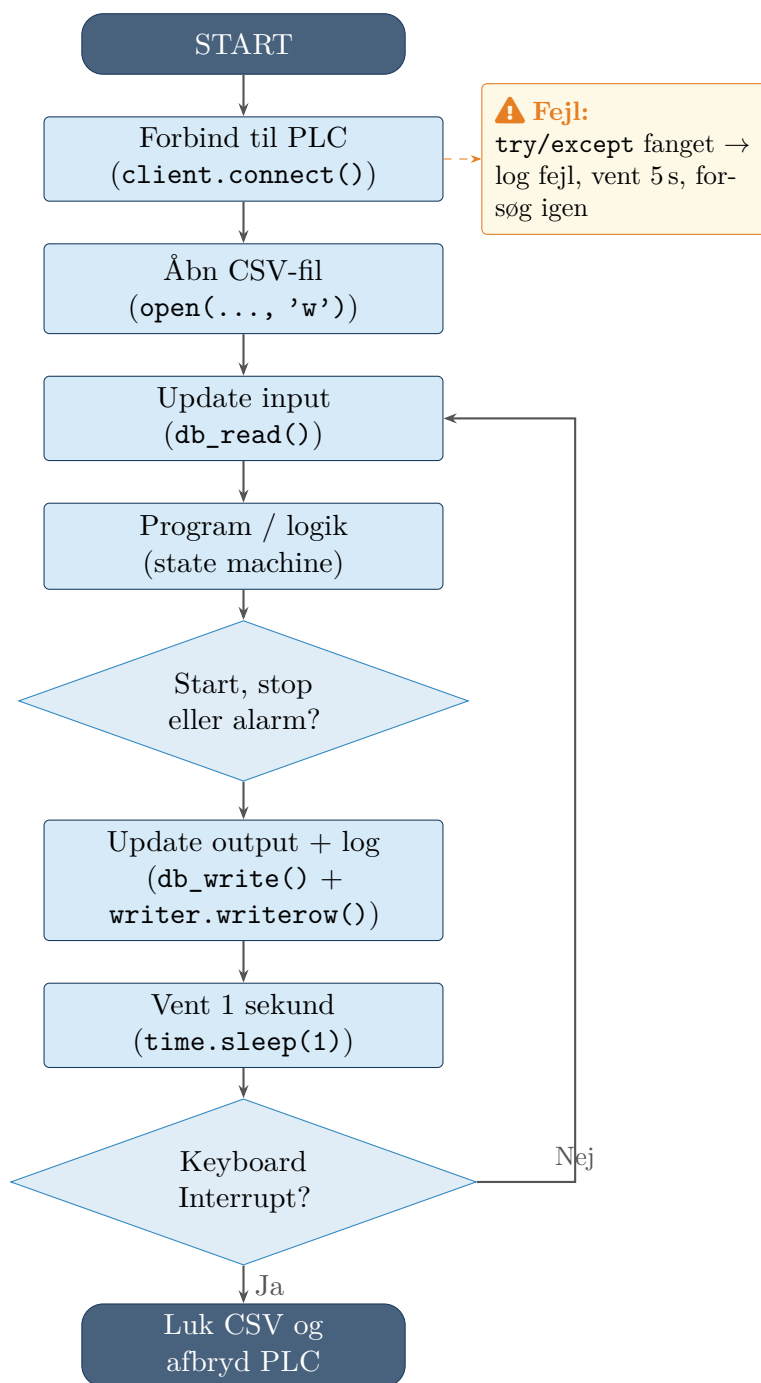
4.1 State machine — Python-script

State maskinen beskriver de mulige tilstande i Python-scriptet og betingelserne for at skifte imellem dem.



⚠ ALARM-state: pumpe stoppes øjeblikkeligt

4.2 Flowchart — Python-hovedløkke



5 | Kodning

Inden du læser koden

Læg mærke til:

- Koden er skrevet **uden funktioner** — alt kører i én sekvens fra toppen
- Hver cyklus følger et simpelt scan-princip: update input → program/logik → update output og log
- CSV-filnavnet dannes automatisk af opstartstidspunktet (NK02)
- State maskinen ligger i program/logik-delen som en **if/elif**-blok
- Snap7-fejl fanges med **try/except** — skriptet stopper ikke (K05)

vandstand.py — del 1/4: imports og indstillinger

```
1 import snap7
2 import csv
3 import time
4 from datetime import datetime
5 from snap7.util import get_real, get_int, get_bool, set_bool
6
7 # Indstillinger
8 PLC_IP = "192.168.0.1"
9 RACK = 0 # Rack 0 for S7-1200
10 SLOT = 1 # Slot 1 for S7-1200
11 DB_NR = 1 # Datablok DB1
12 INTERVAL = 1.0 # Sekunder mellem hvert maal
13 RETRY_DELAY = 5.0 # Sekunder mellem genforsøg ved fejl
14
15 # CSV-filnavn dannes automatisk (opfylder NK02)
16 filnavn = datetime.now().strftime("%Y-%m-%d_%H%M%S") + ".csv"
17 "
```

 vandstand.py — del 2/4: forbind og aaben CSV

```
18     # Forbind til PLC
19     client = snap7.client.Client()
20     while not client.get_connected():
21         try:
22             client.connect(PLC_IP, RACK, SLOT)
23             if client.get_connected():
24                 print("Forbundet til PLC.")
25         except Exception as e:
26             print("Fejl ved forbindelse:", e)
27             time.sleep(RETRY_DELAY)
28
29     # Aaben CSV-fil og skriv kolonneoverskrifter
30     with open(filnavn, "w", newline="", encoding="utf-8") as
31         csv_fil:
32         writer = csv.writer(csv_fil)
33         writer.writerow(["timestamp", "state", "vandstand_pct",
34                         "pumpe_koerer", "alarmkode"])
35
36     # Start-tilstand for state machine
37     state = "IDLE"
```


 vandstand.py — del 3/4: update input og program/logik

```
38     try:
39         while True:
40             try:
41                 # 1. Update input: laes seneste data fra PLC
42                 data = client.db_read(DB_NR, 0, 8)
43
44                 # Udtaek vaerdier fra bytearray
45                 vandstand = get_real(data, 2)          # DBX2: vandstand
46                     0-100 %
47                 pumpe_paa = get_bool(data, 1, 0)      # DBX1 bit0: pumpe
48                     koerer
49                 alarmkode = get_int(data, 6)          # DBX6: 0=OK, 1=
50                     Overflow
51
52                 # 2. Program/logik: beregn naeste state og kommandoer
53                 start_cmd = False
54                 stop_cmd = False
55
56                 if state == "IDLE":
57                     if alarmkode != 0:
58                         state = "ALARM"
59                     elif vandstand < 20.0:
60                         start_cmd = True
61                         state = "FILL"
62
63                 elif state == "FILL":
64                     if alarmkode != 0:
65                         state = "ALARM"
66                     elif vandstand >= 80.0:
67                         stop_cmd = True
68                         state = "IDLE"
69
70                 elif state == "ALARM":
71                     stop_cmd = True
72                     state = "RESET"
73
74                 elif state == "RESET":
75                     if alarmkode == 0:
76                         state = "IDLE"
```



vandstand.py — del 4/4: update output, log og fejlhaandtering

```
77         # 3. Update output og log
78         cmd = client.db_read(DB_NR, 0, 1)
79         set_bool(cmd, 0, 0, start_cmd)
80         set_bool(cmd, 0, 1, stop_cmd)
81         client.db_write(DB_NR, 0, cmd)
82
83         ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
84         writer.writerow([ts, state, f"{vandstand:.1f}",
85                         int(pumpe_paa), alarmkode])
86         csv_fil.flush()
87
88         print(f"{ts} {state} Vandstand: {vandstand:.1f}%")
89
90         # 4. Vent foer naeste cyklus
91         time.sleep(INTERVAL)
92
93     except Exception as e:
94         print("Snap7-fejl:", e)
95
96         ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
97         writer.writerow([ts, "COMM_ERROR", "", "", 2])
98         csv_fil.flush()
99
100        client.disconnect()
101        time.sleep(RETRY_DELAY)
102
103        while not client.get_connected():
104            try:
105                client.connect(PLC_IP, RACK, SLOT)
106                if client.get_connected():
107                    print("Forbundet til PLC igen.")
108            except Exception as reconnect_error:
109                print("Fejl ved genforbindelse:", reconnect_error)
110                time.sleep(RETRY_DELAY)
111
112    except KeyboardInterrupt:
113        print("Stoppet af bruger (Ctrl+C)")
114
115    finally:
116        if client.get_connected():
117            client.disconnect()
118        print(f>Data gemt i: {filnavn}")
```

6 | Testdokumentation

6.1 Factory Acceptance Test (FAT)

FAT udføres på labben, inden systemet evt. flyttes til driftsmiljøet. Her **verificeres** kravene ét for ét mod deres acceptkriterier.

Inden testen

Sørg for at PLC-programmet er indlæst og kørende, og at DB1 er oprettet med den korrekte struktur (se afsnit 3.2). Kontrollér at netværksforbindelsen til PLC er tilgængelig (ping 192.168.0.1).

Krav	Testbetingelse	Forventet resultat	Faktisk resultat	Status
K01	Sæt WaterLevel (DBX2) til 15,0 i PLC-watch	Script skifter til FILL, sender Start-Pump, Q0.0 går HIGH	(udfyldes)	P / F
K02	Kør script med level stigende — sæt DBX2 til 82,0	Script skifter til IDLE, sender Stop-Pump, Q0.0 går LOW	(udfyldes)	P / F
K03	Sæt DBX6 til 1 (overflow) mens system er i FILL	Script skifter til ALARM, stopper pumpen, logger ALARM	(udfyldes)	P / F
K04	Lad script køre i 10 sek., åbn CSV	CSV-fil indeholder 10 rækker med korrekte kolonner og timestamps	(udfyldes)	P / F
K05	Frakobl netværkskabel mens script kører	Script logger COMM_ERROR, vent 5 s, genopretter forbindelsen	(udfyldes)	P / F

6.1.1 Performance-kontrol (NK01)

Mål tidsforskel mellem to på hinanden følgende timestamps i CSV-filen. Alle intervaller skal ligge under 1,5 s.

Måling	Timestamp	Næste	Δt (s)	OK?
1	(udfyldes)	(udfyldes)		
2	(udfyldes)	(udfyldes)		
3	(udfyldes)	(udfyldes)		

6.2 Accepttest (UAT)

Accepttest — brugerens perspektiv

Accepttesten **validerer**, at den færdige løsning fungerer som tiltænkt i brugerens virkelighed. Den udføres typisk af en person, der ikke har skrevet koden, og her tester man ikke hvert krav separat, men om systemet samlet set er forståeligt og anvendeligt.

- ✓ Start scriptet som bruger og kontrollér at det er tydeligt, at anlægget er forbundet og logger data
- ✓ Gennemfør en normal driftssekvens og vurder om pumpeanlæggets opførsel giver mening set fra operatørens side
- ✓ Simulér en alarm og vurder om fejltilstanden er let at forstå og håndtere
- ✓ Stop scriptet med Ctrl+C og kontrollér at logfilen bagefter kan bruges som dokumentation for forløbet

7 | Eksempel på CSV-output

Et typisk uddrag fra den genererede CSV-fil ser sådan ud:

 2026-04-05_091532.csv

```
1 timestamp,state,vandstand_pct,pumpe_koerer,alarmkode
2 2026-04-05 09:15:32,IDLE,42.3,0,0
3 2026-04-05 09:15:33,IDLE,38.7,0,0
4 2026-04-05 09:15:34,IDLE,19.2,0,0
5 2026-04-05 09:15:35,FILL,19.2,1,0
6 2026-04-05 09:15:36,FILL,24.1,1,0
7 2026-04-05 09:15:37,FILL,31.8,1,0
8 ...
9 2026-04-05 09:16:44,FILL,80.3,1,0
10 2026-04-05 09:16:45,IDLE,80.3,0,0
11 2026-04-05 09:17:03,ALARM,55.0,0,1
12 2026-04-05 09:17:04,RESET,55.0,0,1
13 2026-04-05 09:17:09,IDLE,55.0,0,0
```

Hvis du vil visualisere loggen efter testen, kan du bruge et separat script:

 plot_vandstand.py

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 df = pd.read_csv("2026-04-05_091532.csv", parse_dates=["
    timestamp"])
5 df.set_index("timestamp", inplace=True)
6
7 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6), sharex
    =True)
8
9 ax1.plot(df["vandstand_pct"], color="steelblue", label="
    Vandstand (%)")
10 ax1.axhline(20, color="orange", linestyle="--", label="Start
    -graense (20%)")
11 ax1.axhline(80, color="red",      linestyle="--", label="Stop-
    graense (80%)")
12 ax1.set_ylabel("Vandstand [%]")
13 ax1.legend()
14
15 ax2.step(df.index, df["pumpe_koerer"],
16         where="post", color="green", label="Pumpe koerer")
17 ax2.set_ylabel("Pumpe")
18 ax2.set_xlabel("Tid")
19 ax2.legend()
20
21 plt.tight_layout()
22 plt.savefig("vandstand_plot.png", dpi=150)
23 plt.show()
```

8 | Opsummering — processen kort

V-model trin	Hvad vi lavede	Output
Requirements	K01–K05 med acceptkriterier for verifikation	Tabel i afsnit 2
System Design	Blokdiagram, DB1-struktur	Afsnit 3
Architecture Design	DB-adresser, dataflow	Tabel i afsnit 3.2
Module Design	State machine, flowchart	Afsnit 4
Coding	Python-script med docstrings	Afsnit 5
Unit Test	<i>Ikke vist — test enkelt-funktioner isoleret</i>	—
Integration Test	Factory Acceptance Test: PLC ↔ Python	Tabel i afsnit 6.1
System Test	Fuld sekvens: IDLE→FILL→ALARM→RESET	Afsnit 6.1
Acceptance Test	UAT: validering af brug og helhed	Afsnit 6.2

For simplificering så er der ikke lavet egentlige unit tests da kode ofte bliver inddelt i funktioner og derfor kan dette være svært at lave unit tests på.

★ Det er sådan vi gerne vil have det

Dette dokument er et **eksempel** på, hvad vi forventer af jer. Jeres eget projekt behøver ikke være et pumpeanlæg — men processen skal være den samme:

- Krav **før** kode. Kravene styrer, hvad I tester.
- Diagrammer **før** kode. State machine og flowchart tegnes først.
- Koden **følger** designet — ikke omvendt.
- Test er **planlagt** og dokumenteret — ikke bare “det virker”.